CS342L – Advanced Database Management Systems Lab

Lab 5 – Installation Requirements

Before starting Lab 5, install the required Python packages for Django and PostgreSQL connectivity.

1. Open the Terminal

Open **VS Code** and open the **Terminal**:

Terminal → New Terminal

Make sure you are working inside your Lab 5 project folder.

2. Install Django

Django is the Python web framework that will be used to build the backend application.

Run the following command in the VS Code terminal:

pip install django

After installation, verify that Django was installed successfully:

django-admin --version

If the installation is successful, the terminal will display the installed Django version.

3. Install PostgreSQL Database Driver

Django needs a PostgreSQL driver to communicate with the PostgreSQL database.

Install psycopg2-binary using:

pip install psycopg2-binary

This package allows Python/Django to connect to PostgreSQL.

4. Install Both Packages Together

Alternatively, both packages can be installed with a single command:

pip install django psycopg2-binary

5. Verify the Installations

Run the following commands:

python -m django --version

and:

pip show psycopg2-binary

You should see the installed Django version and information about the psycopg2-binary package.

Required Packages

Package	Purpose
django	Provides the Django web framework
psycopg2-binary	Allows Django/Python to connect to PostgreSQL

2. Create the Django Project and Application

After installing Django and the PostgreSQL driver, create a Django project that will be used to connect with the existing dvdrental database.

Step 1: Open the VS Code Terminal

Open the terminal in VS Code:

Terminal → New Terminal

Navigate to the folder where you want to create your Lab 5 project.

Step 2: Create the Django Project

Run:

```
django-admin startproject dvdrental_site
```

This creates a Django project named **dvdrental_site**.

The project structure will look similar to:

```
dvdrental_site/
│
├── manage.py
│
└── dvdrental_site/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

Important Files

- **manage.py** – used to run Django commands.
- **settings.py** – contains project configuration, including the database settings.
- **urls.py** – defines the URLs/routes of the application.

Step 3: Move into the Project Folder

Run:

cd dvdrental_site

You should now be inside the Django project directory.

Step 4: Create the Django Application

Create an application named rentals:

python manage.py startapp rentals

The project will now contain:

```
dvdrental_site/
│
├── manage.py
│
├── dvdrental_site/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
└── rentals/
    ├── migrations/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
```

```
├── models.py
├── tests.py
└── views.py
```

Project vs Application

In this lab:

- **dvdrental_site** is the Django **project**.
- **rentals** is the Django **application** inside the project.

The project contains the overall configuration, while the application contains the functionality related to the rental system.

Step 5: Add the Application to Django

Open the following file in VS Code:

dvdrental_site/settings.py

Find:

INSTALLED_APPS = [

Add 'rentals' to the list:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'rentals',
]
```

Save the file.

Step 6: Check the Django Project

In the terminal, run:

python manage.py check

If everything is configured correctly, you should see:

System check identified no issues (0 silenced).

Note

At this stage, Django has been created, but it is **not yet connected to the dvdrental PostgreSQL database**.

The next step is to configure the PostgreSQL database connection in:

dvdrental_site/settings.py

3. Configure the PostgreSQL Database Connection

In this step, Django will be connected to the existing **dvdrental PostgreSQL database** created in the previous labs.

Step 1: Open settings.py

In VS Code, open:

dvdrental_site/settings.py

Find the default DATABASES section:

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.sqlite3',
    'NAME': BASE_DIR / 'db.sqlite3',
  }
}
```

Replace it with the PostgreSQL configuration below.

Step 2: Configure PostgreSQL

Use:

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.postgresql',
    'NAME': 'dvdrental',
    'USER': 'postgres',
    'PASSWORD': 'YOUR_POSTGRES_PASSWORD',
    'HOST': 'localhost',
    'PORT': '5432',
  }
```

}

Replace the Following Values

Setting	Value
ENGINE	PostgreSQL database engine
NAME	dvdrental
USER	Your PostgreSQL username
PASSWORD	Your PostgreSQL password
HOST	localhost
PORT	Your PostgreSQL port

Important: If PostgreSQL is running on the default port, use:

5432

If PostgreSQL on your computer is configured to use another port, enter that port instead.

Step 3: Save the File

Save:

dvdrental_site/settings.py

Make sure that:

- The database name is correct.
- The PostgreSQL username is correct.
- The password is correct.
- The host is correct.
- The port matches the PostgreSQL server.

Step 4: Check the Django Configuration

Open the VS Code terminal and run:

python manage.py check

If the Django configuration is valid, you should see:

System check identified no issues (0 silenced).

Note: This command checks the Django configuration, but it does not fully prove that Django can connect to PostgreSQL.

4. Test the Database Connection

Now verify that Django can actually connect to the dvdrental database.

Run:

python manage.py shell

You should see the Python/Django shell.

Then enter:

from django.db import connection

Now test the connection:

connection.ensure_connection()

If there is no error, Django successfully connected to PostgreSQL.

Step 5: Test with a SQL Query

Run the following inside the Django shell:

```
with connection.cursor() as cursor:
  cursor.execute("SELECT COUNT(*) FROM customer;")
  print(cursor.fetchone())
```

You should get a result similar to:

(599,)

The exact number may vary depending on your database.

This query checks whether Django can access the existing customer table in the dvdrental database.

Expected Result

If the connection is successful:

Django → PostgreSQL → dvdrental → customer table

At this point, Django is successfully connected to the database used in the previous labs.

Common Connection Errors

1. password authentication failed

The PostgreSQL username or password is incorrect.

Check:

'USER': 'postgres',

'PASSWORD': 'YOUR_POSTGRES_PASSWORD',

2. connection refused

The PostgreSQL server may not be running, or the port may be incorrect.

Check the PostgreSQL service and verify the port.

3. database "dvdrental" does not exist

The database name is incorrect or the dvdrental database has not been created.

4. ModuleNotFoundError: No module named 'psycopg2'

Install the PostgreSQL driver:

pip install psycopg2-binary

Important

Do **not** create a new SQLite database for this lab. The purpose of this lab is to connect Django to the **existing PostgreSQL dvdrental database** used in the previous labs.